Special Participation E (Google NotebookLM) - RNNs and SSMs

Anders Vestrum (3041972833)

Introduction: This report explores how Google NotebookLM, an AI-enhanced learning platform, can serve as a guided tutor for understanding complex concepts in deep learning. Specifically, I focused on State-Space Models (SSMs) and their relationship to Recurrent Neural Networks (RNNs). This is a topic covered in the EECS 182 course on Deep Neural Networks (Fall 2025). I found these topics conceptually difficult at first, particularly the computational transition from sequential recurrence to parallel convolution and the mathematical structure of the S4 model.

To examine whether AI could assist in clarifying such advanced material, I used NotebookLM in Learning Mode, which actively questions the learner to probe understanding. I uploaded several learning sources, including lecture notes, homework assignments on RNNs and SSMs, and the original S4: Efficiently Modeling Long Sequences with Structured State Spaces paper. Alongside this interaction, I created a video overview to introduce the topic at a high level and a mind map to visually organize the relationships among RNNs, SSMs, and attention mechanisms.

The objective of this project was twofold: first, to evaluate how effectively AI tools like NotebookLM can scaffold the learning process for complex theoretical material; and second, to identify best practices for integrating such tools into self-directed study.

Key findings: Through this experiment, I found that NotebookLM was particularly effective in fostering conceptual reasoning and active engagement. The chatbot's questioning style pushed me to articulate the differences between RNNs and SSMs, reinforcing understanding through explanation rather than passive reading. For instance, by prompting me to express SSM computation as a convolution ($y = K * u$) and to recall how the FFT accelerates training from $O(L^2)$

to O(L log L), the system helped me internalize why SSMs are inherently more parallel and efficient.

The dialogue also clarified how the S4 model's Diagonal Plus Low-Rank (DPLR) parameterization and the Woodbury Matrix Identity enable efficient kernel computation, making it possible to process very long sequences while maintaining stability. This guided explanation bridged high-level intuition with mathematical detail, which I had struggled to connect before.

Overall, the interaction demonstrated how AI-led learning can complement traditional lectures by enabling students to actively converse with material, test understanding, and refine mental models through dialogue rather than memorization.

Recommendations to other students: Based on this experience, I would recommend the following workflow for using NotebookLM or similar AI-assisted tools effectively:

1. **Start with an overview (video or audio).** Begin with a brief conceptual video or NotebookLM-generated summary to form an initial mental map of the topic. This step helps orient you before diving into the technical details.
2. **Customize the prompt.** When starting a learning session, clearly specify what aspects you want the AI to focus on (e.g., "Explain the efficiency of S4 compared to RNNs," or "Focus on mathematical intuition rather than implementation"). This ensures the responses are tailored to your learning goals.
3. **Engage in dialogue.** Use Learning Mode or a similar questioning feature to let the AI quiz and challenge your understanding. Treat it like a Socratic tutor—respond thoughtfully, and ask the AI to justify its claims.
4. **Consolidate with quizzes or summaries.** NotebookLM can generate short quizzes based on your sources, which are helpful for immediate feedback and reinforcement.
5. **Use visual supports.** Complement AI discussions with visual tools such as mind maps or diagrams to connect concepts across sources.
6. **Reflect critically.** After the session, review where the AI was strong and where it misled you. Annotating hallucinations or oversimplifications helps build critical awareness of AI limitations.

In my view, this workflow allows AI to act as an interactive co-learner rather than a passive information provider. The combination of overview → guided discussion

→ self-assessment builds a layered understanding that traditional reading alone often fails to achieve.

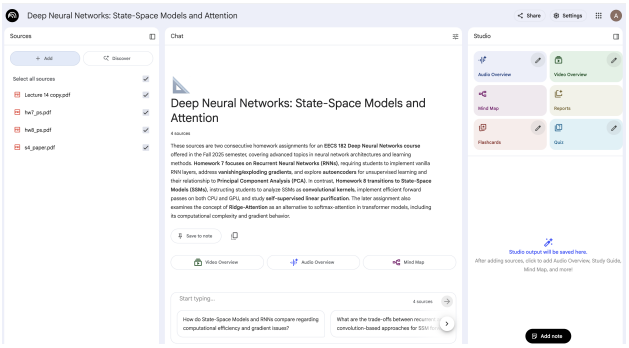

Figure 1: Dashboard of the NotebookLM

Evaluation of the discussion with "learning guide" From my standpoint, the dialogue was strong overall but it missed a small yet important point: it stated the FFT cost as $N \cdot L \log L$, which I realized was wrong since standard FFT-based convolution parallelizes across time, so the asymptotics should have been $L \log L$, and the lack of parallelization should have been called out explicitly. That slip aside, I appreciated how it guided me with good questions, helped me connect RNN sequential bottlenecks to SSM convolutional parallelism and tools like diagonalization and the Woodbury identity, and most notably introduced Ridge-Attention, clarifying how it reframes self-attention compared to the transformer's softmax and prompting connections I hadn't made before. Before using NotebookLM, I struggled to connect the math of the S4 model to its intuition. After the session, I could explain the DPLR parameterization and Woodbury identity and why they are useful.

NotebookLM: These sources are two consecutive homework assignments for an EECS 182 Deep Neural Networks course offered in the Fall 2025 semester, covering advanced topics in neural network architectures and learning methods. Homework 7 focuses on Recurrent Neural Networks (RNNs), requiring students to implement vanilla RNN layers, address vanishing/exploding gradients, and explore autoencoders for unsupervised learning and their relationship to Principal Component Analysis (PCA). In contrast, Homework 8 transitions to State-Space Models (SSMs), instructing students to analyze SSMs as convolutional kernels, implement efficient forward passes on both CPU and GPU, and study self-supervised linear purification. The later assignment also examines the concept of Ridge-Attention as an alternative to softmax-attention in transformer models, including its computational complexity and gradient behavior.

Student: I want to learn about these sources using Learning Guide

NotebookLM: These sources cover a rich area of deep learning, focusing mainly on sequence modeling through Recurrent Neural Networks (RNNs), State-Space Models (SSMs), and self-supervised learning methods. How much do you currently know about these topics, and what is your high-level goal for exploring this material? For example, are you preparing for a test, or are you looking to understand a specific model like S4?

Student: I'm familiar with the basic computational difference between RNNs and SSMs. In RNNs, the hidden state is updated sequentially each step depends on the previous one which prevents full parallelization and makes training on long sequences expensive. In contrast, State-Space Models like S4 reformulate sequence processing so that the entire sequence can be handled as a convolution with a learned kernel, allowing parallel computation across time steps. I also understand that SSMs compute this kernel efficiently by using structured matrix parameterizations (like diagonalization of a transition matrix) that let them compute powers of that matrix in a more numerically stable and efficient way.

NotebookLM: That's an excellent summary of the core computational concepts! You already grasp the crucial distinction: RNNs rely on recurrence, while SSMs leverage convolution for parallelism during training. Let's dive deeper into the specifics using the sources to understand why SSMs are more efficient.

The efficiency difference stems from how training computation is performed.

1. How does the RNN's recurrent nature prevent efficiency?
2. How does the SSM's convolutional nature enable efficiency?

Let's start with the first point. Based on the source material, what is the key limitation of traditional RNNs regarding parallelism during training?

Student: Each RNN layer in the time dimension depends on the previous hidden state, so you can't compute step t until you've finished step $t-1$. This sequential dependency makes RNNs inherently non-parallelizable across time steps. In contrast, SSMs compute hidden states as linear combinations of matrix powers and input signals, so they don't rely on previous states in time and can compute all steps in parallel.

NotebookLM: Exactly right! The sequential update of the hidden state in RNNs (h_t depends on h_{t-1}) prevents full parallelization across time steps during training. This long dependency chain also contributes to vanishing or exploding gradients.

Now, focusing on SSMs. You're correct that their computation is fundamentally different from recurrence. Based on the sources, how is the output sequence y of a discrete-time SSM formally computed, and what operation enables massive parallelization?

Student: I think we can rewrite the SSM as a convolution of the input with a kernel like $y = K * u$. That lets us compute all outputs at once using the FFT instead of going state-by-state like in RNNs.

NotebookLM: Exactly! That's the fundamental computational advantage of State-Space Models like S4. The output sequence y can be expressed as:

$$y = K * u,$$

where the kernel K is:

$$K = (CB, CAB, CA^2B, \ldots, CA^{L-1}B).$$

This convolutional form enables massive parallelization across time steps, something RNNs cannot do.

The efficiency comes from two steps:

1. Parallel Computation of Output (Given K): Using FFTs to compute $y = K * u$.
2. Efficient Computation of the Kernel K: Making kernel computation itself fast and memory-efficient.

Let's start with the first one. If the kernel K is already known, how does computing $y = K * u$ with FFTs improve efficiency?

Student: Since K has length L (the sequence length), direct convolution takes $O(L^2)$ time. But using FFTs reduces it to $O(L \log L)$, because convolution in the time domain becomes element-wise multiplication in the frequency domain. That's where the parallelism comes in.

NotebookLM: Exactly. The FFT transforms the convolution from the time domain to the frequency domain, where it's just element-wise multiplication which is much faster and fully parallelizable. The critical path length, or the minimum time needed to complete computation, drops from $O(L)$ for sequential RNNs to roughly $O(\log L)$ for FFT-based convolution.

Now, computing the convolution is efficient, but generating the kernel $K = (CA^kB)$ naively still takes $O(N^2L)$. How would you compute those terms in practice?

Student: We could compute them iteratively. Each step applies A once to a vector, which costs $O(N^2)$. Doing this for L steps gives $O(N^2L)$ overall, which is pretty slow.

NotebookLM: Exactly. That's the bottleneck S4 addresses. It does so by parameterizing A in a special way - a *Normal Plus Low-Rank* (NPLR) or *Diagonal Plus Low-Rank* (DPLR) form:

$$A = D + PQ^\top.$$

This makes it possible to diagonalize A efficiently. Can you reason how this structure speeds up the computation of all A^kB terms?

Student: If A can be diagonalized as $A = V\Lambda V^{-1}$, then $A^k = V\Lambda^k V^{-1}$. Since Λ is diagonal, we can compute Λ^k by raising its diagonal entries elementwise which is much faster and easy to parallelize.

NotebookLM: Perfect. That's the main idea. This structure reduces kernel computation from $O(N^2L)$ to about $O(NL \log L)$.

Now, when A has a DPLR structure, meaning a diagonal matrix plus a low-rank correction - which mathematical identity helps simplify matrix inversions like $(I - zA)^{-1}$?

Student: That would be the Woodbury Matrix Identity, right? It is used to efficiently invert diagonal plus low-rank matrices. Isnt this also what is shown in the HW8 problem 1f?

NotebookLM: Excellent insight - yes, that's precisely the right connection. The *Woodbury Matrix Identity* is the key mathematical tool that allows S4 (and related SSM formulations) to efficiently handle matrices of the form

$$A = D + pp^\top.$$

In this setup:

$$(I - zA)^{-1} = (I - zD - zpp^\top)^{-1},$$

which would normally require inverting a dense $N \times N$ matrix for each complex frequency z. But with Woodbury, this inversion simplifies dramatically:

$$(I - zA)^{-1} = (I - zD)^{-1} - (I - zD)^{-1}p\Big(1 + zp^\top(I - zD)^{-1}p\Big)^{-1}p^\top(I - zD)^{-1}.$$

Because D is diagonal (or easily invertible), and the rank-one term $pp^\top$ only introduces a scalar correction, this expression can be computed in $\mathcal{O}(N)$ instead of $\mathcal{O}(N^3)$ time.

This is exactly how S4 efficiently evaluates the SSM generating function in the frequency domain - it turns a potentially expensive matrix inverse into simple elementwise operations and one small scalar inverse.

So your intuition is right on point: the **Woodbury identity** is what bridges the DPLR parameterization and S4's efficient kernel computation.

Based on the sources, when applying this identity, what does the core problem reduce to after the low-rank correction is handled? (Hint: Think about what S4 says the diagonal case is equivalent to.)

Student: So after applying Woodbury, the heavy matrix operations collapse into these elementwise Cauchy terms. That explains why the method is both stable and efficient.

NotebookLM: Exactly. The Cauchy kernel representation allows efficient, parallel computation of the entire kernel K in $O(\tilde{N} + \tilde{L})$ time — making S4 scalable to long sequences.

Now, stepping back — RNNs suffer from vanishing or exploding gradients. Why do SSMs, especially S4, mitigate this?

Student: Because SSMs compute the sequence as a single convolution, so the gradient path is much shorter. And S4's structured diagonalization keeps eigenvalues in a stable range, preventing gradient explosion or decay.

NotebookLM: Exactly right. The convolutional formulation ensures stable gradient flow.

Now, let's connect this to attention mechanisms. How do SSMs relate to attention?

Student: Both model long-range dependencies, but attention learns input-dependent kernels (via queries and keys), while SSMs have fixed, learned linear dynamics. SSMs are less flexible but much faster and more memory-efficient.

NotebookLM: Well said. Homework 8 also introduces Ridge-Attention, which replaces the softmax with a ridge regression solution:

$$w^* = (A^\top A + \lambda I)^{-1} A^\top y.$$

This formulation keeps computation stable and linear, conceptually similar to how SSMs rely on linear structure for efficient sequence modeling.

Interesting connection, right? If Ridge-Attention replaces softmax with a ridge regression solution, how do you think this changes the way attention distributes importance across tokens compared to standard softmax attention? (Hint: Consider what the regularization term λI does to the solution, and how that might affect sparsity and stability.)

Student: How does the ridge attention differ from the transformer in terms if its formula and learnable parameters?

NotebookLM: That's an excellent follow-up question that targets the fundamental design differences. Ridge-Attention (RA) is a substitution for traditional Softmax-Attention (SA) that relies entirely on linear algebra rather than probability.

Ridge-Attention is built directly upon the ridge regression closed-form solution, where the prediction is based on finding an optimal weight vector w^*. The closed-form solution is given by

$$w^* = (A^\top A + \lambda I)^{-1} A^\top y.$$

In the context of self-attention, the keys play the role of the data matrix A, the values correspond to the output vector y (which becomes a matrix in the multidimensional case), and the query acts as the new test input x for which a prediction is computed via ${w^*}^\top x$.

Softmax-Attention, in contrast, can be viewed as a softened version of a nearest-neighbor model. It computes weights using a softmax function, leading to a probabilistic weighting where the output is a linear combination of the values. The formulation is given by

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

In a typical Transformer architecture, the Query (Q), Key (K), and Value (V) matrices are learned projections. When applying backpropagation through a Ridge-Attention layer, several components can successfully pass gradients and are therefore treated as learnable parameters: the keys, the values, the queries, and potentially the ridge coefficient λ if it is viewed as a learnable parameter for the layer.

The key distinction between the two mechanisms is that Ridge-Attention uses a direct matrix inverse (with the regularization term λ) to solve for the weights, whereas Softmax-Attention uses the softmax function to compute probabilistic weights.

Since we previously discussed the efficiency gains of State-Space Models (SSMs), an interesting next step could be to explore the computational cost of Ridge-Attention compared to standard attention, or to analyze how the ridge coefficient λ influences the learned purification matrix.